

Architecture Decision Records

ADR-001 a ADR-005 — Formuloo OS

ADR-001 — Architecture : Microservices purs

Contexte

Formuloo OS est une suite ERP intégrant 6 domaines fonctionnels : Auth, RH, Comptabilité, CRM, Stock et Analytics. Chaque domaine a des cycles de vie, des exigences de performance et des besoins de scalabilité différents. Il était nécessaire de choisir un style architectural capable de gérer cette diversité tout en garantissant la maintenabilité et l'évolutivité du système.

Options considérées

Option	Avantages	Inconvénients
Monolithe classique	Simple à déployer, moins de complexité réseau	Couplage fort, scalabilité limitée, point de défaillance unique
Monolithe modulaire	Compromis pragmatique, plus simple à gérer	Modules liés, scalabilité limitée par module
Microservices purs (choisi)	Indépendance totale, scalabilité fine, isolation des pannes, déploiement indépendant	Complexité réseau, nécessite API Gateway et Message Broker

Décision

Nous adoptons une architecture microservices purs : chaque domaine est un service indépendant avec sa propre base PostgreSQL, son propre conteneur Docker et son propre cycle de déploiement.

Justification

- Chaque microservice peut être scalé indépendamment selon la charge
- Isolation des pannes : si le service Stock tombe, HR et Compta continuent
- Déploiement indépendant : mise à jour CRM sans impacter les autres services
- Principe Database per Service : isolation totale des données entre domaines
- Alignement avec les pratiques modernes pour les ERP Saas africains

Conséquences

- Mise en place obligatoire d'un API Gateway pour centraliser le routage
- Implémentation d'un Message Broker pour la communication asynchrone
- Keycloak requis pour le SSO centralise entre tous les services
- Pipeline CI/CD GitLab configure indépendamment par service

ADR-002 — Authentification : JWT + Keycloak SSO

Contexte

Formuloo OS comporte 6 microservices indépendants. Les utilisateurs doivent se connecter une seule fois et accéder à tous les modules sans se reconnecter (SSO). L'architecture multi-tenant exige que chaque token transporte le `tenant_id` pour garantir l'isolation des données entre PME.

Options considérées

Option	Avantages	Inconvénients
Sessions Django classiques	Simple à implémenter, natif Django	Non adapté aux Microservices, pas de SSO natif, sessions centralisées
JWT seul (sans Keycloak)	Léger, stateless, facile à valider	Pas de gestion centralisée, révocation complexe, pas de SSO natif
JWT + Keycloak (choisi)	SSO natif, OAuth2/OIDC standard, RBAC avancée, révocation facile	Infrastructure supplémentaire à déployer

Décision

Nous utilisons Keycloak comme serveur SSO centralisé (OAuth2/OIDC). Keycloak émet des tokens JWT contenant `userId`, `tenant_id` et rôles. Chaque microservice valide le JWT de façon autonome.

Configuration retenue

- Access Token : durée de vie 15 minutes
- Refresh Token : durée de vie 7 jours
- Blacklisting actif (tokens révoqués stockés dans Redis)
- Algorithme RS256 (clé asymétrique)
- Contenu JWT : `userId`, `tenant_id`, rôles, `exp`, `iat`

Conséquences

- Keycloak déployé comme service indépendant dans docker-compose
- Chaque service Django configure avec la clé publique Keycloak
- Redis utilise pour le blacklisting des tokens révoqués
- Middleware Django personnalisée pour extraire `tenant_id` et rôles du JWT

ADR-003 — Visionnement de l'API : URL versioning

Contexte

Formuloo OS expose une API publique consommée par le Frontend Angular et des clients tiers. L'API va évoluer au fil du temps. Il est impératif que ces évolutions ne cassent pas les consommateurs existants. Une stratégie de visionnement claire doit être définie dès le départ.

Options considérées

Option	Avantages	Inconvénients
URL versioning /api/v1/ (choisi)	Explicite, visible dans les logs, facile à router, lisible	URL plus longue
Header versioning Accept: v1	URLs propres	Invisible dans les logs, difficile à tester, non standard
Query param ?version=1	Simple à implémenter	Pollue les URLs, cache compliqué, non standard

Décision

Nous adoptons le versionnement par URL : /api/v1/{domaine}/{ressource}/. La version est toujours présente dans l'URL et constitue la première information de routage après /api/.

Exemples

- /api/v1/hr/employees/ → version stable actuelle
- /api/v2/hr/employees/ → future version avec breaking changes
- /api/v1/compta/factures/ → stable
- /api/v1/crm/clients/ → stable

Politique de deprecation

- Annoncer avec le header HTTP Deprecation: true
- Maintenir l'ancienne version pendant minimum 3 mois
- Documenter les migrations dans le CHANGELOG
- Supprimer uniquement après accord des deux équipes

ADR-004 — Base de données : PostgreSQL + Redis

Contexte

Formuloo OS gère des données métier complexes et relationnelles dans un contexte multi-tenant. Il faut une base de données robuste pour la persistance des données, et un système de cache performant pour les sessions, le rate limiting et les réponses fréquentes.

Options considérées — Base principale

Option	Avantages	Inconvénients
PostgreSQL (choisi)	ACID, support JSON, schémas multiples, excellent avec Django ORM, open source	Nécessite plus de configuration que SQLite
MySQL / MariaDB	Populaire, performant	Moins bon support JSON, transactions moins robustes
MongoDB	Flexible pour schémas variables	Pas adapté aux données relationnelles ERP, pas de transactions ACID complètes

Options considérées — Cache

Option	Avantages	Inconvénients
Redis (choisi)	Ultra-rapide, Pub/Sub, queues, sessions, TTL natif, blacklisting JWT	Données en mémoire (volatiles sans persistance)
Memcached	Simple, rapide	Pas de Pub/Sub, pas de structures avancées
Cache Django mémoire	Zéro configuration	Non partage entre instances, perdu au redémarrage

Décision

PostgreSQL 15 comme base principale (une instance par microservice — principe Database per Service) et Redis 7 comme cache partagé pour les sessions, le rate limiting et le blacklisting JWT.

Conséquences

- 6 instances PostgreSQL indépendantes dans docker-compose
- 1 instance Redis partagée entre tous les services
- Django-redis configure comme Backend de cache dans chaque service
- Support JSON/JSONB PostgreSQL utilise pour les champs flexibles d'Analytics

ADR-005 — Approche Contract-First avec OpenAPI 3

Contexte

Le projet Formuloo OS est développé en binôme : Backend (Django REST) et Frontend (Angular) en parallèle sur des sprints de 2 semaines. Sans contrat clair défini au préalable, les risques de désaccord d'intégration sont élevés. Il faut une approche permettant au Frontend de développer sans attendre le Backend.

Options considérées

Option	Avantages	Inconvénients
Code-First (coder puis documenter)	Démarrage rapide, moins de planification	Documentation incomplète, désaccords fréquents, intégration difficile
Contract-First OpenAPI 3 (choisi)	Contrat valide avant le code, Frontend peut mocker, conformité vérifiable	Phase de conception initiale plus longue
Documentation manuelle	Simple	Toujours désynchronisée, non vérifiable, source de conflits

Décision

Nous adoptons l'approche Contract-First : le contrat OpenAPI 3 est rédigé et valide par les deux équipes AVANT toute implémentation. drf-spectacular génère la spécification. Schemathesis vérifie la conformité en CI/CD.

Flux de travail

- Backend + Frontend définissent ensemble le contrat YAML
- Contrat versionne sur gitlab.formuloo.com/formuloo-contracts
- Frontend génère son client TypeScript via ng-openapi-gen
- Frontend développe contre Prism (mock server)
- Backend implémente les endpoints Django
- drf-spectacular génère la spec OpenAPI depuis le code
- Schemathesis compare spec générée vs contrat initial en CI/CD

Conséquences

- drf-spectacular installe et configure dans chaque service Django
- Schemathesis intègre dans le pipeline GitLab CI/CD
- Prism déployé comme mock server pour le Frontend
- Toute modification de contrat passe par une MR validée par les deux équipes